VIPER Programming Language

CFG Practice: Calculator & Logical Problem-Solving Programs

Purpose

These programs are designed as practice inputs for expanding and testing the initial VIPER CFG. They cover assignment, expressions, conditions, Boolean logic, loops, input/output, and functions.

> **Keywords used:** HOLD, RELEASE, fang, venom, scale, coil, hiss, shed, strike, shift, slither, coilrun, stalk, lurk, escape, slide, nest, fangback, sense, hissout.

1. Basic Calculator

Arithmetic expressions, initialization, identifiers, integers, assignment, and output.

```
HOLD

fang a = 20;
fang b = 5;

fang sum = a + b;
fang difference = a - b;
fang product = a * b;
fang quotient = a / b;
fang remainder = a % b;

hissout("Sum:", sum);
hissout("Difference:", difference);
hissout("Product:", product);
hissout("Quotient:", quotient);
hissout("Remainder:", remainder);

RELEASE
```

2. Calculator with Division Check

Input, arithmetic, relational conditions, and if/else.

```
HOLD

fang first;
fang second;

sense(first);
sense(second);

fang sum = first + second;
fang difference = first - second;
fang product = first * second;

strike (second != 0) {
    fang quotient = first / second;
    hissout("Quotient:", quotient);
}
slither {
    hissout("Cannot divide by zero");
}

hissout("Sum:", sum);
hissout("Difference:", difference);
hissout("Product:", product);

RELEASE
```

3. Even or Odd

Modulus, equality, conditionals, and output.

```
HOLD

fang number;
sense(number);
```

```
strike (number % 2 == 0) {
    hissout("Even");
}
slither {
    hissout("Odd");
}

RELEASE
```

4. Positive, Negative, or Zero

strike, shift, slither, and relational operators.

```
HOLD

fang number;
sense(number);

strike (number > 0) {
    hissout("Positive");
}
shift (number < 0) {
    hissout("Negative");
}
slither {
    hissout("Zero");
}

RELEASE
```

5. Largest of Three Numbers

Relational operators combined with logical AND and multiple branches.

```
HOLD

fang a;
fang b;
fang c;

sense(a);
sense(b);
sense(c);

strike (a >= b && a >= c) {
    hissout("A is largest");
}
shift (b >= a && b >= c) {
    hissout("B is largest");
}
slither {
    hissout("C is largest");
}

RELEASE
```

6. Grade Calculator

A chain of shift branches and ordered relational conditions.

```
HOLD

fang marks;
sense(marks);

strike (marks >= 80) {
    hissout("Grade A");
}
shift (marks >= 70) {
    hissout("Grade B");
}
shift (marks >= 60) {
    hissout("Grade C");
}
shift (marks >= 50) {
    hissout("Grade D");
}
slither {
```

```
    hissout("Fail");
}

RELEASE
```

7. Leap Year

Nested parentheses, modulus, equality, inequality, AND, and OR.

```
HOLD

fang year;
sense(year);

strike ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0))) {
    hissout("Leap Year");
}
slither {
    hissout("Not a Leap Year");
}

RELEASE
```

8. Menu Calculator

Nested conditionals, arithmetic expressions, and branch chains.

```
HOLD

fang choice;
fang a;
fang b;

sense(choice);
sense(a);
sense(b);

strike (choice == 1) {
    hissout("Result:", a + b);
}
shift (choice == 2) {
    hissout("Result:", a - b);
}
shift (choice == 3) {
    hissout("Result:", a * b);
}
shift (choice == 4) {
    strike (b != 0) {
        hissout("Result:", a / b);
    }
    slither {
        hissout("Cannot divide by zero");
    }
}
slither {
    hissout("Invalid choice");
}

RELEASE
```

9. Factorial

Loop condition, repeated assignment, multiplication, and an accumulator.

```
HOLD

fang n;
fang result = 1;
fang i = 1;

sense(n);

stalk (i <= n) {
    result = result * i;
    i = i + 1;
}

hissout("Factorial:", result);
```

```
RELEASE
```

10. Sum of First N Numbers

Looping, arithmetic assignment, and a running total.

```
HOLD

fang n;
fang i = 1;
fang total = 0;

sense(n);

stalk (i <= n) {
    total = total + i;
    i = i + 1;
}

hissout("Total:", total);

RELEASE
```

11. Prime Number Check

Boolean variables, nested conditions, loops, modulus, and escape.

```
HOLD

fang number;
fang divisor = 2;
hiss prime = true;

sense(number);

strike (number < 2) {
    prime = false;
}
slither {
    stalk (divisor * divisor <= number) {
        strike (number % divisor == 0) {
            prime = false;
            escape;
        }
        divisor = divisor + 1;
    }
}

strike (prime == true) {
    hissout("Prime");
}
slither {
    hissout("Not Prime");
}

RELEASE
```

12. Boolean Logic

Boolean literals, AND, NOT, and conditional logic.

```
HOLD

hiss loggedIn = true;
hiss verified = false;

strike (loggedIn && !verified) {
    hissout("Verification required");
}
shift (loggedIn && verified) {
    hissout("Access granted");
}
slither {
    hissout("Login required");
}

RELEASE
```

13. Nested Conditional

Nested strike/slither blocks and Boolean comparison.

```
HOLD

fang age;
hiss student;

sense(age);
sense(student);

strike (age >= 18) {
    strike (student == true) {
        hissout("Adult student");
    }
    slither {
        hissout("Adult non-student");
    }
}
slither {
    hissout("Minor");
}

RELEASE
```

14. Function Example

Function declaration, parameters, local initialization, function call, and fangback.

```
HOLD

nest fang add(fang a, fang b) {
    fang result = a + b;
    fangback result;
}

fang x = 10;
fang y = 15;

fang answer = add(x, y);

hissout("Answer:", answer);

RELEASE
```

CFG Coverage Checklist

CFG area	Covered by
Program	All examples: HOLD ... RELEASE
Declaration	Examples 2, 3, 4, 5, 6
Initialization	Examples 1, 7, 9, 12
Assignment	Examples 9, 10, 11
Arithmetic	+, -, *, /, %
Relational	<, >, <=, >=, ==, !=
Logical	&&, ||, !
Conditionals	strike, shift, slither
Nested blocks	Examples 8, 11, 13
Loops	stalk examples
Loop control	escape
Boolean	true, false
Functions	nest, fangback
I/O	sense, hissout

Next CFG step: use these programs as test strings. For each program, trace the grammar from through , then verify that expressions, conditions, and blocks terminate in the expected terminals. Check the resulting grammar for left recursion, left factoring, and ambiguity.